



Engineering EVM Precompiles for Cryptographic Primitives

Zach Kelling

Lux Industries

2025

Abstract

An EVM precompile is a native function callable from Solidity at a designated address, executing outside the EVM interpreter at near-native speed. Precompiles enable cryptographic operations that would be prohibitively expensive in EVM bytecode. This paper teaches the engineering of precompiles from first principles: the EVM precompile interface, address space conventions, input/output ABI design, gas cost modeling based on empirical benchmarks, determinism requirements, error handling, and testing methodology. We present all 20 precompiles deployed on Lux Network’s C-Chain, covering BLS12-381, BN254, batch secp256k1, post-quantum signatures (ML-DSA, FALCON), FHE operations, Warp message verification, and NTT. For each precompile, we provide the Go implementation, gas cost analysis, security considerations, and Solidity interface contract. This is a complete guide for blockchain engineers adding native cryptographic capabilities to EVM-compatible chains.

Contents

1	What Is a Precompile	4
1.1	The EVM Execution Model	4
1.2	Precompiles: Native Extensions	4
1.3	Why Precompiles Instead of Libraries	4
2	The Precompile Interface	4
2.1	Go Interface	4
2.2	Registration	5
2.3	Address Space Convention	5
3	Input/Output ABI	6
3.1	Design Principles	6
3.2	Example: BLS12-381 Pairing Check	6
4	Gas Cost Modeling	7
4.1	Principle: Gas Reflects Wall-Clock Time	7
4.2	Calibration Method	7
4.3	Gas Cost Table	7
5	Determinism Requirements	7
5.1	Why Determinism Is Non-Negotiable	7
5.2	Sources of Non-Determinism	8
5.3	Determinism Testing	8
6	Error Handling	8
6.1	Precompile Error Semantics	8
6.2	Input Validation	8
6.3	Denial-of-Service Resistance	9
7	The 20 Precompiles	10
8	Testing Methodology	10
8.1	Four-Level Testing	10
9	Security Considerations	11
9.1	Subgroup Checks	11
9.2	Gas Griefing	11
9.3	Timing Side Channels	11

10 Solidity Interface Contracts	11
11 The DEX Adapter Precompile (0x9010)	12
11.1 A CLOB Facade, Not an AMM	12
11.2 Inert by Default	12
11.3 Native LUX as <code>address(0)</code>	13
11.4 Content-Addressed Idempotency	13
11.5 Why This Belongs in the Precompile Layer	13
12 Conclusion	13

1 What Is a Precompile

1.1 The EVM Execution Model

The EVM executes smart contracts as sequences of opcodes (bytecodes). Each opcode has a fixed gas cost. Opcodes provide basic operations: arithmetic, memory access, storage, control flow. There are no opcodes for complex cryptographic operations like pairing checks or post-quantum signature verification.

1.2 Precompiles: Native Extensions

A precompile is a special contract address (typically in the range 0x0001–0x00ff for Ethereum, 0x0200–0x02ff for Lux extensions) where a `CALL` or `STATICCALL` invokes a native Go function instead of interpreting EVM bytecode.

From Solidity’s perspective, a precompile call looks like a normal external call:

Listing 1: Calling a precompile from Solidity

```
1 // Call the BLS12-381 pairing precompile at 0x0200
2 (bool success, bytes memory result) = address(0x0200).staticcall(
3     abi.encodePacked(g1Point, g2Point)
4 );
5 require(success, "pairing_check_failed");
6 bool pairingResult = abi.decode(result, (bool));
```

1.3 Why Precompiles Instead of Libraries

Implementing BLS12-381 pairing in Solidity would cost approximately 50M gas (most of the block gas limit). The same operation as a precompile costs 113,000 gas—a 440x reduction. The reason: EVM opcodes have overhead per operation (gas metering, stack manipulation, memory management), and cryptographic operations require millions of field multiplications that incur this overhead for each one.

2 The Precompile Interface

2.1 Go Interface

Every precompile implements a standard Go interface:

Listing 2: The precompile interface

```
1 // PrecompiledContract is the interface for native contracts.
2 type PrecompiledContract interface {
3     // RequiredGas returns the gas cost for the given input.
4     // Must be deterministic: same input -> same gas.
5     RequiredGas(input []byte) uint64
6
7     // Run executes the precompile with the given input.
8     // Returns output bytes and an error.
9     // Must be deterministic: same input -> same output.
10    Run(input []byte) ([]byte, error)
11 }
```

Two methods, both deterministic:

1. **RequiredGas**: Called before execution to check if the caller has enough gas. Must not perform expensive computation—it should be $O(1)$ or $O(\text{input length})$.

2. **Run:** Performs the actual computation. Returns output bytes or an error. On error, all gas is consumed (precompile failure is an out-of-gas condition).

2.2 Registration

Precompiles are registered in the VM configuration:

Listing 3: Registering a precompile

```

1 var PrecompiledContractsLux = map[common.Address]PrecompiledContract{
2     // Standard Ethereum precompiles (0x01 - 0x09)
3     common.BytesToAddress([]byte{0x01}): &ecRecover{},
4     common.BytesToAddress([]byte{0x02}): &sha256hash{},
5     // ...
6
7     // Lux extensions (0x0200 - 0x02ff)
8     common.BytesToAddress([]byte{0x02, 0x00}): &bls12381Pairing{},
9     common.BytesToAddress([]byte{0x02, 0x01}): &bls12381G1Add{},
10    common.BytesToAddress([]byte{0x02, 0x02}): &bls12381G1Mul{},
11    common.BytesToAddress([]byte{0x02, 0x03}): &bls12381MapFp{},
12    common.BytesToAddress([]byte{0x02, 0x10}): &bn254Pairing{},
13    common.BytesToAddress([]byte{0x02, 0x20}): &batchEcRecover{},
14    common.BytesToAddress([]byte{0x02, 0x30}): &dilithiumVerify{},
15    common.BytesToAddress([]byte{0x02, 0x31}): &falconVerify{},
16    common.BytesToAddress([]byte{0x02, 0x40}): &fheEncrypt{},
17    common.BytesToAddress([]byte{0x02, 0x41}): &fheAdd{},
18    common.BytesToAddress([]byte{0x02, 0x42}): &fheMul{},
19    common.BytesToAddress([]byte{0x02, 0x50}): &warpVerify{},
20    common.BytesToAddress([]byte{0x02, 0x60}): &nttForward{},
21    common.BytesToAddress([]byte{0x02, 0x61}): &nttInverse{},
22    // ... (20 total)
23 }
```

2.3 Address Space Convention

Range	Owner	Purpose
0x0001–0x0009	Ethereum	Standard precompiles
0x000a–0x00ff	Ethereum	Reserved for EIPs
0x0100–0x01ff	Reserved	Chain-specific
0x0200–0x020f	Lux	BLS12-381 operations
0x0210–0x021f	Lux	BN254 extensions
0x0220–0x022f	Lux	Batch operations
0x0230–0x023f	Lux	Post-quantum signatures
0x0240–0x024f	Lux	FHE operations
0x0250–0x025f	Lux	Warp/cross-chain
0x0260–0x026f	Lux	Polynomial/NTT
0x9000–0x90ff	Lux	Application adapters (DEX)

Table 1: Precompile address space allocation. The 0x90xx application range holds non-cryptographic adapters; the DEX CLOB adapter at 0x9010 is documented in Section 11.

3 Input/Output ABI

3.1 Design Principles

1. **Fixed-size fields:** All inputs use fixed-width encoding (32 bytes for scalars, 48 bytes for G1 points, etc.). No length-prefixed variable fields.
2. **Left-padded:** Scalars are left-padded with zeros to 32 bytes, matching Solidity ABI conventions.
3. **Concatenated:** Multiple inputs are concatenated without separators. The precompile deduces structure from the total input length.
4. **Error = revert:** Invalid inputs cause the precompile to return an error, which the EVM interprets as a revert consuming all provided gas.

3.2 Example: BLS12-381 Pairing Check

Listing 4: BLS12-381 pairing input/output format

```
1 // Input: k pairs of (G1, G2) points, concatenated
2 // Each pair: 128 bytes (48 for G1 compressed + 96 for G2 compressed,
3 //           padded to 128 for alignment)
4 // Total input: k * 128 bytes
5 //
6 // Output: 32 bytes
7 // 0x00...01 if pairing check passes (product of pairings == 1)
8 // 0x00...00 if pairing check fails
9
10 func (c *bls12381Pairing) Run(input []byte) ([]byte, error) {
11     if len(input) == 0 || len(input)%128 != 0 {
12         return nil, errInvalidInput
13     }
14
15     k := len(input) / 128
16     pairs := make([]PairingPair, k)
17
18     for i := 0; i < k; i++ {
19         chunk := input[i*128 : (i+1)*128]
20
21         g1, err := decodeG1(chunk[:48])
22         if err != nil {
23             return nil, err
24         }
25         if !g1.IsOnCurve() || !g1.IsInSubgroup() {
26             return nil, errNotOnCurve
27         }
28
29         g2, err := decodeG2(chunk[48:128])
30         if err != nil {
31             return nil, err
32         }
33         if !g2.IsOnCurve() || !g2.IsInSubgroup() {
34             return nil, errNotOnCurve
35         }
36
37         pairs[i] = PairingPair{G1: g1, G2: g2}
38     }
39
40     result := multiPairing(pairs)
```

```

41     output := make([]byte, 32)
42     if result.IsOne() {
43         output[31] = 1
44     }
45     return output, nil
46 }

```

4 Gas Cost Modeling

4.1 Principle: Gas Reflects Wall-Clock Time

Gas costs must reflect actual computation time to prevent denial-of-service attacks. If a precompile is underpriced, an attacker can consume node CPU without paying proportional gas. If overpriced, legitimate use is discouraged.

4.2 Calibration Method

1. Benchmark the precompile on reference hardware (AMD EPYC 7713, which runs Lux Network validators).
2. Measure wall-clock time for representative inputs.
3. Compute gas per nanosecond using the baseline: `ecrecover` costs 3,000 gas and takes approximately $26\mu\text{s}$ on reference hardware, giving a rate of approximately 0.115 gas/ns.
4. Gas cost = execution time (ns) $\times$ 0.115, rounded up to the nearest 1,000.

4.3 Gas Cost Table

Precompile	Time (μs)	Gas	Gas/ns
BLS12-381 pairing (1 pair)	1,700	113,000	0.066
BLS12-381 G1 add	2.4	500	0.208
BLS12-381 G1 scalar mul	340	45,000	0.132
BN254 pairing (1 pair)	820	65,000	0.079
Batch ecrecover (100 sigs)	12,000	180,000	0.015
ML-DSA-65 verify	300	35,000	0.117
FALCON-512 verify	180	22,000	0.122
FHE encrypt	450	55,000	0.122
FHE add	12	2,000	0.167
FHE multiply	850	100,000	0.118
Warp message verify	95	12,000	0.126
NTT-256 forward	3.1	500	0.161
NTT-256 inverse	3.3	500	0.152

Table 2: Gas costs calibrated to execution time on AMD EPYC 7713.

All gas/ns ratios fall within the range $[0.015, 0.208]$, a factor of 14x variation. The target is $0.115 \pm 2x$. Batch ecrecover is intentionally underpriced to incentivize batching (it replaces 100 individual ecrecover calls at 300,000 gas with 180,000 gas for the batch).

5 Determinism Requirements

5.1 Why Determinism Is Non-Negotiable

Every validator must compute identical state transitions. If a precompile produces different outputs on different hardware, validators disagree on the state root, causing a consensus split.

5.2 Sources of Non-Determinism

1. **Floating-point arithmetic:** Never use floating point in precompiles. All arithmetic is integer or modular.
2. **Map iteration order:** Go map iteration is randomized. Never iterate over maps in precompile code.
3. **Goroutines:** Concurrent execution with shared state is non-deterministic. Precompiles must be single-threaded.
4. **System calls:** Time, random number generation, file I/O. Never use these.
5. **Compiler optimizations:** Different Go compiler versions may produce different results for certain edge cases. Pin the Go version and test across platforms.

5.3 Determinism Testing

We run every precompile test on three platforms (AMD64, ARM64, WASM) and verify identical outputs. Any divergence is a critical bug.

Listing 5: Determinism test

```
1 func TestDeterminism(t *testing.T) {
2     // Golden outputs computed on reference platform
3     for _, tc := range goldenTestCases {
4         output, err := precompile.Run(tc.Input)
5         if err != nil {
6             t.Fatalf("unexpected error: %v", err)
7         }
8         if !bytes.Equal(output, tc.ExpectedOutput) {
9             t.Fatalf("determinism violation:\n"+
10                 "input: %x\n"+
11                 "got: %x\n"+
12                 "expected: %x",
13                 tc.Input, output, tc.ExpectedOutput)
14         }
15     }
16 }
```

6 Error Handling

6.1 Precompile Error Semantics

When a precompile returns an error, the EVM:

1. Consumes all gas provided to the call.
2. Returns no output data.
3. Reverts state changes made within the call (but the gas is still consumed).

6.2 Input Validation

Every precompile validates inputs before processing:

Listing 6: Input validation pattern

```
1 func (c *dilithiumVerify) Run(input []byte) ([]byte, error) {
2     // Check total length
3     if len(input) < PubKeyLen+SigLen+32 {
4         return nil, errInvalidInputLength
5     }
6 }
```



```

7 // Decode public key
8 pk, err := decodeDilithiumPubKey(input[:PubKeyLen])
9 if err != nil {
10     return nil, errInvalidPublicKey
11 }
12
13 // Decode signature
14 sig, err := decodeDilithiumSig(input[PubKeyLen : PubKeyLen+SigLen])
15 if err != nil {
16     return nil, errInvalidSignature
17 }
18
19 // Message is the remainder
20 msg := input[PubKeyLen+SigLen:]
21
22 // Verify
23 valid := dilithium.Verify(pk, msg, sig)
24
25 output := make([]byte, 32)
26 if valid {
27     output[31] = 1
28 }
29 return output, nil
30 }

```

6.3 Denial-of-Service Resistance

The `RequiredGas` function must accurately bound the computation cost. For variable-cost operations (e.g., pairing with k pairs), the gas scales linearly:

Listing 7: Variable gas cost for pairing

```

1 func (c *bls12381Pairing) RequiredGas(input []byte) uint64 {
2     if len(input) == 0 || len(input)%128 != 0 {
3         return 0 // Will fail in Run
4     }
5     k := uint64(len(input)) / 128
6     return 43000 + k*70000 // Base + per-pair cost
7 }

```

7 The 20 Precompiles

Addr	Name	Gas	Function
0x0200	BLS12-381 Pairing	113K/pair	Multi-pairing check
0x0201	BLS12-381 G1 Add	500	Elliptic curve addition
0x0202	BLS12-381 G1 Mul	45K	Scalar multiplication
0x0203	BLS12-381 Map-to-G1	5.5K	Hash to curve
0x0204	BLS12-381 G2 Add	800	G2 point addition
0x0205	BLS12-381 G2 Mul	128K	G2 scalar multiplication
0x0206	BLS12-381 Map-to-G2	23K	Hash to G2
0x0207	BLS12-381 Aggregate	12K	Aggregate BLS signatures
0x0210	BN254 Pairing	65K/pair	BN254 multi-pairing
0x0211	BN254 G1 Mul Batch	6K/point	Batched scalar mul
0x0220	Batch ecrecover	1.8K/sig	Batch secp256k1 verify
0x0230	ML-DSA Verify	35K	Dilithium-3 verification
0x0231	FALCON Verify	22K	FALCON-512 verification
0x0240	FHE Encrypt	55K	TFHE encryption
0x0241	FHE Add	2K	Ciphertext addition
0x0242	FHE Multiply	100K	Ciphertext multiplication
0x0243	FHE Decrypt	45K	Ciphertext decryption
0x0250	Warp Verify	12K	Cross-chain message verify
0x0260	NTT Forward	500	Forward NTT-256
0x0261	NTT Inverse	500	Inverse NTT-256

Table 3: All 20 Lux Network precompiles with address and gas cost.

8 Testing Methodology

8.1 Four-Level Testing

1. **Known-Answer Tests (KATs):** Test against NIST/standard test vectors. Verifies correctness for specific inputs.
2. **Fuzz Tests:** Random inputs to find panics, incorrect results, and non-determinism.
3. **Integration Tests:** Call precompiles from Solidity via Foundry. Verifies the full path from Solidity through the EVM to the native code and back.
4. **Cross-Platform Tests:** Run identical tests on AMD64, ARM64, and WASM. Any output difference is a critical bug.

Listing 8: Integration test for post-quantum verification

```

1 function test_dilithiumVerifyFromSolidity() public {
2     // Known-good signature from NIST test vectors
3     bytes memory input = abi.encodePacked(
4         dilithiumPubKey,
5         dilithiumSignature,
6         testMessage
7     );
8
9     (bool success, bytes memory result) = address(0x0230).staticcall(
10         input);
11     assertTrue(success, "precompile call failed");
12     uint256 verified = abi.decode(result, (uint256));

```

```

13     assertEq(verified, 1, "valid_signature_should_verify");
14 }
15
16 function test_dilithiumRejectsInvalidSig() public {
17     bytes memory invalidSig = new bytes(dilithiumSignature.length);
18     // Zero signature should not verify
19     bytes memory input = abi.encodePacked(
20         dilithiumPubKey,
21         invalidSig,
22         testMessage
23     );
24
25     (bool success, bytes memory result) = address(0x0230).staticcall(
26         input);
27     assertTrue(success, "precompile_should_not_revert_on_invalid_sig");
28
29     uint256 verified = abi.decode(result, (uint256));
30     assertEq(verified, 0, "invalid_signature_should_not_verify");
31 }

```

9 Security Considerations

9.1 Subgroup Checks

For BLS12-381 and BN254, input points must be checked for subgroup membership. Accepting points outside the prime-order subgroup enables small-subgroup attacks that can forge signatures or break pairing assumptions.

9.2 Gas Griefing

If `RequiredGas` underestimates the true cost, an attacker can craft inputs that consume more CPU time than the gas price reflects, slowing down validators. We benchmark worst-case inputs (maximum-cost curve operations, maximum-length batch operations) and set gas costs to the worst case.

9.3 Timing Side Channels

Precompile execution time must not depend on secret data. For verification precompiles, this is straightforward: the input (public key, signature, message) is public. For FHE precompiles that handle encrypted data, we use constant-time implementations.

10 Solidity Interface Contracts

We provide Solidity libraries for type-safe precompile access:

Listing 9: Solidity interface library

```

1 library PQCrypto {
2     address constant DILITHIUM = address(0x0230);
3     address constant FALCON = address(0x0231);
4
5     function verifyDilithium(
6         bytes memory pubKey,
7         bytes memory signature,
8         bytes memory message

```

```

9      ) internal view returns (bool) {
10          (bool ok, bytes memory result) = DILITHIUM.staticcall(
11              abi.encodePacked(pubKey, signature, message)
12          );
13          if (!ok || result.length != 32) return false;
14          return abi.decode(result, (uint256)) == 1;
15      }
16
17      function verifyFalcon(
18          bytes memory pubKey,
19          bytes memory signature,
20          bytes memory message
21      ) internal view returns (bool) {
22          (bool ok, bytes memory result) = FALCON.staticcall(
23              abi.encodePacked(pubKey, signature, message)
24          );
25          if (!ok || result.length != 32) return false;
26          return abi.decode(result, (uint256)) == 1;
27      }
28  }

```

11 The DEX Adapter Precompile (0x9010)

The twenty precompiles documented above are *cryptographic* primitives: pure functions of their input bytes with no external state. Lux adds one precompile of a different kind — an *application adapter* — at address 0x9010, in the application range 0x9000–0x90ff reserved above the cryptographic block. It is the EVM-side entry point to the Lux DEX, and its engineering is instructive precisely because it is not a pure function: it is a deterministic facade over an external matching chain.

11.1 A CLOB Facade, Not an AMM

0x9010 exposes a Uniswap-V4-PoolManager-style singleton interface — pool keys, hooks, flash accounting — but it is a **central-limit-order-book (CLOB) facade**, not an automated market maker. A call does not move along a bonding curve; it relays an order to the D-Chain matching VM (Lux Lightspeed DEX [7]) and reports the resulting fill. The V4-shaped surface is retained so that existing V4-aware routers and Solidity integrators compose unchanged, while the execution semantics underneath are price-time priority CLOB matching rather than constant-product swaps.

This is deliberately distinct from the plain Solidity AMMs Lux also ships: the V2 (constant-product) and V3 (concentrated-liquidity) pools are ordinary contracts with no precompile involvement. The precompile exists only for the CLOB path, where on-chain bytecode cannot match a limit-order book at the throughput the D-Chain achieves natively.

11.2 Inert by Default

The precompile is **inert unless the validator is configured with a dex-zap-endpoint**. With no endpoint configured, 0x9010 is registered but performs no relay: it is dormant, exactly as a reserved-but-unwired entry would be. This keeps the public `luxd` binary free of any dependency on the private D-Chain — the adapter is present in the precompile table but does nothing until an operator explicitly wires it to a matching endpoint. Determinism is preserved trivially in the inert state, and the configured state is made deterministic by the idempotency discipline below.

11.3 Native LUX as address(0)

The pool-key encoding follows the V4 convention that the native asset is the zero address. A pool key referencing `address(0)` denotes native LUX; ERC-20 securities use their token address. This removes the wrapped-native special case from the adapter ABI: every asset, native or token, is a 20-byte address in the pool key, and `address(0)` routes to the D-Chain’s native-balance path.

11.4 Content-Addressed Idempotency

A relay precompile faces a hazard a pure precompile does not: the same EVM call may be re-executed (block re-org, validator restart, speculative execution) and must not double-submit the underlying order. The adapter keys idempotency on a **content-addressed transaction identifier** — the `txID` is a hash of the order’s canonical bytes, so an identical order produces an identical `txID`, and the D-Chain deduplicates on it. Re-execution of the same EVM call relays the same `txID`, which the matcher recognizes as already-seen and does not re-match. The precompile thus presents exactly-once semantics to the EVM despite being a relay rather than a pure computation — the same determinism contract every other precompile honors, achieved by content-addressing instead of statelessness.

11.5 Why This Belongs in the Precompile Layer

A CLOB cannot be matched in EVM bytecode at competitive cost — the same argument that justifies the cryptographic precompiles. The adapter keeps the EVM as the settlement and composition surface (V4-compatible, hook-friendly) while delegating the latency-critical matching to a chain engineered for it. The full D-Chain VM architecture — matcher-at-Verify, `zapdb` commit, crash-restart durability, and the GPU CLOB matcher behind it — is specified in the Lux Lightspeed DEX paper [7]; this section documents only the EVM-side adapter and the engineering discipline (inert-by-default, `address(0)` native, content-addressed idempotency) that makes a stateful relay safe to expose as a precompile.

12 Conclusion

EVM precompiles are the mechanism by which blockchain VMs gain new cryptographic capabilities without changing the EVM instruction set. Engineering them correctly requires attention to determinism, gas pricing, input validation, error handling, and cross-platform testing. Done well, precompiles enable practical use of advanced cryptography (post-quantum signatures, FHE, pairing-based proofs) in smart contracts at gas costs comparable to basic operations.

Our 20 precompiles provide Lux Network’s C-Chain with a comprehensive cryptographic toolkit. Each one follows the same engineering discipline: deterministic execution, gas cost calibrated to wall-clock time, subgroup and input validation, and four levels of testing.

Canonical c-abi migration (cevm v0.46.0–v0.47.0)

`cevm v0.46.0` (Phase 5b) rewired `cevm/lib/cevm_precompiles/bls.cpp` and `kzg.cpp` (564 lines of in-tree blst) into thin `extern "C"` wiring through the brand-neutral `luxcpp/crypto/bls` and `luxcpp/crypto/kzg` c-abi (EIP-2537 BLS12-381 G1/G2 add/mul/msm + pairing_check + map_fp/fp2, EIP-4844 KZG point-evaluation). Production binaries (`libcevm_precompiles.a`, `libevm.dylib`, `libevm-kernel-metal.a`) link zero blst symbols, asserted in CI by `cevm/test/unittests/no_blst` on every build (`nm + otool -L + nm -a` all clean). `cevm v0.47.0` (commit `aea6ba08`) migrated 7 additional algorithms to forward-shim headers calling the canonical `luxcpp/crypto` c-abi, removing 2 107 lines of duplicated implementation. Pairing plus keccak / secp256k1 are deferred

pending relative-include flatten and legacy API consumer migration. The subgroup policy (`SubgroupPolicy::CheckAndReject` for validator pubkeys, `ClearIfHashToCurve` only for hash-to-curve outputs) lives in the canonical c-abi and is exercised against blst as a test-only oracle pinned at `luxcpp/crypto/bls/test/cmake/`.

References

- [1] Reitwiessner, C. “EIP-196: Precompiled Contracts for Addition and Scalar Multiplication on the Elliptic Curve alt_bn128.” 2017.
- [2] Reitwiessner, C. “EIP-197: Precompiled Contracts for Optimal Ate Pairing Check on the Elliptic Curve alt_bn128.” 2017.
- [3] Vlasov, A. et al. “EIP-2537: Precompile for BLS12-381 Curve Operations.” 2020.
- [4] Wood, G. “Ethereum: A Secure Decentralised Generalised Transaction Ledger (Yellow Paper).” 2014.
- [5] NIST. “Module-Lattice-Based Digital Signature Standard (ML-DSA).” FIPS 204, 2024.
- [6] Prest, T. et al. “FALCON: Fast-Fourier Lattice-Based Compact Signatures over NTRU.” NIST PQC, 2020.
- [7] Lux Industries, Inc. “Lux Lightspeed DEX: On-Chain Order Book with Sub-Second Finality.” Lux Technical Report.